Piano Analyzer

Finger Tracking and Piano Note Transcription

Technical Report — Original Implementation (October 2026)

Juan Sanchez

October 15, 2026

Challenges and Planned Improvements

The Piano Analyzer program processes recorded piano videos with audio to track fingertips and detect note onsets. MediaPipe provides hand landmarks used to identify fingertip candidates. The output consists of an annotated MP4 video with audio and a standard MIDI file (performance.mid).

The main challenges are camera movement, hidden keys or fingertips, shadows, and complex fast passages, which received limited testing. Across seven development recordings, the program achieved 87.74% precision and 84.19% recall for pitch and onset accuracy. The target precision and recall thresholds were met in aggregate, but the original 30 FPS goal was not reached. Physical key releases are not measured.

To resolve these issues, future work includes collecting separate test recordings with fixed camera positioning, tuning calibration, and testing generalization across unseen clips.

Implementation Evolution

The current implementation replaces the previous NOTE_ON, HELD, and NOTE_OFF detection states because visual tracking struggled to identify key presses and releases reliably. It combines fingertip positions with audio to detect note starts, then estimates note endings for MIDI output. Instead of comparing key brightness, it uses librosa to find note starts and FFT checks to confirm notes; this approach was more accurate during development. Each candidate note also requires fingertip support in at least two nearby frames. OpenCV handles reading video frames, perspective mapping, color conversion, drawing annotations, and writing the annotated video, while MediaPipe provides the hand landmarks.

Keyboard calibration retains manually selected corners and C-key centers while removing automatic line and contour checks. Calibration uses a JSON file without a reference image. The original implementation processes recorded videos with audio over a fixed C3-C7 range and produces an annotated video with audio and a MIDI file. Calibration, audio analysis, and video rendering are separated into individual modules, and MIDI export uses pretty_midi.

Introduction and Functional Requirements

Piano Analyzer extends an earlier hand-tracking prototype to map piano keys, track fingertips, and combine visual observations with audio analysis to detect note pitches and start times. Across seven development recordings, it achieved 87.74% precision, 84.19% recall, and 16.37 FPS pooled processing throughput. Physical key releases and playing dynamics are not measured. I developed the project individually. This report describes the system architecture, implementation, evaluation methodology, performance results, and remaining limitations.

Project Capabilities

Capability	Current Implementation
Types of Input	Prepared constant-frame-rate video containing piano audio.
Piano/Keyboard Mapping	Manually calibrate C3 to C7 (29 white keys, 20 black keys).
Finger Tracking	Tracks up to 2 hands using MediaPipe landmarks 4, 8, 12, 16, and 20. Onset times are confirmed with audio FFT checks. Physical key releases and dynamic velocity are not measured. MIDI notes use a fixed velocity of 64.
Note Recording	Detected pitches and onset times, with set note endings of .20 s or MIDI output.
Types of Output	performance.mid and annotated.mp4 only.

Proposed Outcomes

Outcome	Intended Scope	FPS	Precision	Recall
Minimum	C4-C6 white note keys, hand presence detection, and note logging	15	> 0.70	> 0.30
Target	All 49 keys, fingertip mapping, and MIDI	30	> 0.80	> 0.50
Optimal	Two hand and 3 note chord detection, and verification through DAW	30	> 0.90	> 0.50

Across seven development recordings, the program produced 229 true positives, 32 false positives, and 43 false negatives across 272 reference notes. Precision was 87.74%, recall was 84.19%, and F1 was 85.93%. Combined metrics pool note counts across recordings rather than averaging individual clip percentages.

Machine Vision and Machine Learning Requirements

Keyboard Region

The program uses manual four-corner calibration and C-key anchors to establish perspective mapping into normalized keyboard coordinates. Black keys are mapped relative to local white key spacing.

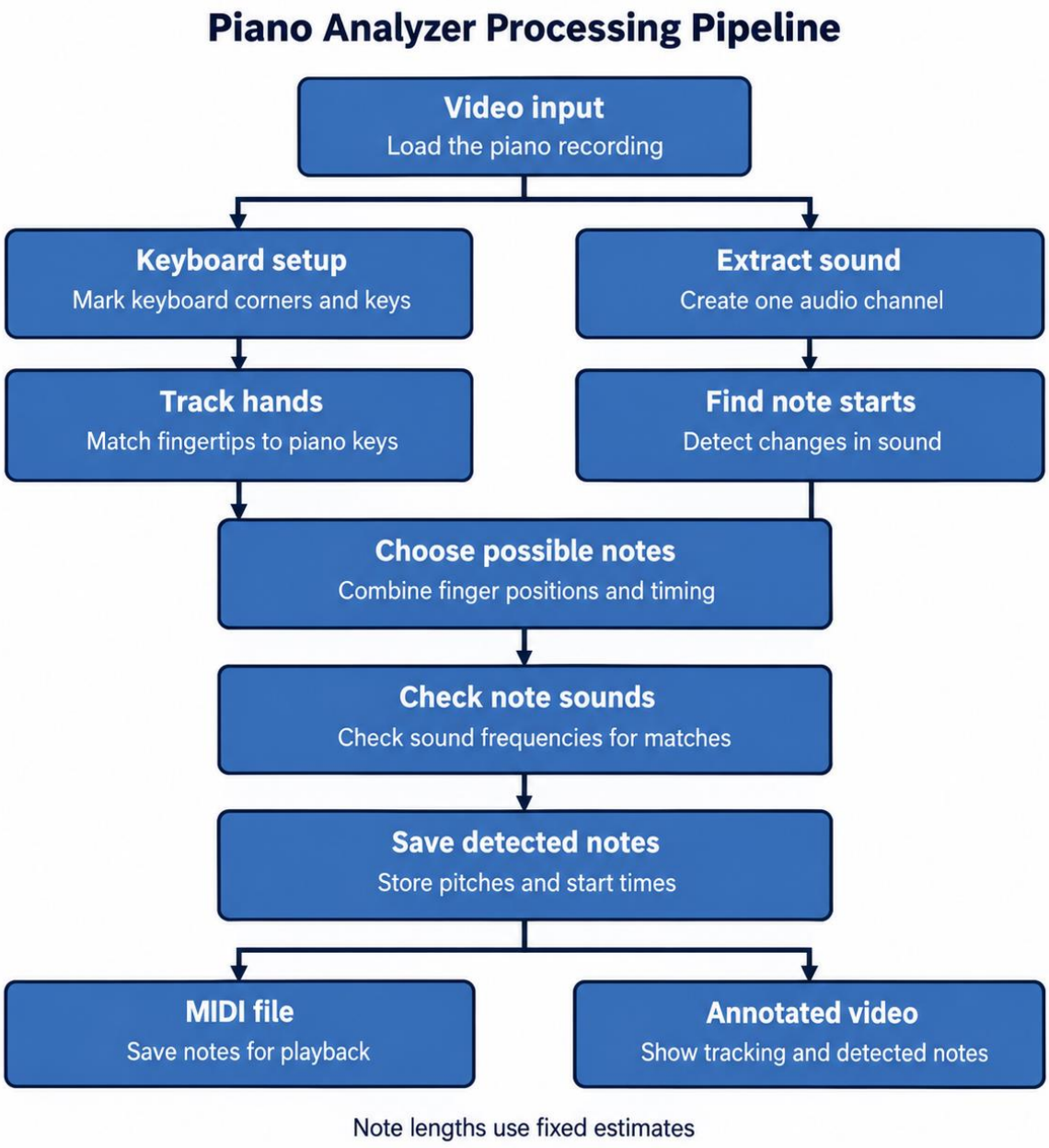
Hand Landmarks

The program uses MediaPipe's GestureRecognizer in VIDEO mode to track hand landmarks. Thumb and fingertips correspond to landmarks 4, 8, 12, 16, and 20 for up to two hands. Image coordinates are transformed into the keyboard view to identify candidate keys.

Press Classification

Press detection combines visual candidate gathering with audio confirmation. Fingertip observations within 0.10 seconds before or after each audio onset are counted; candidate pitches require support in at least two distinct frames. Candidate pitches then undergo FFT pitch checks comparing frequency spectrum changes before and after the attack to confirm the fundamental and harmonics.

Functional Design Overview



Reported accuracy measures exact pitch matches within ±0.10 seconds of a labeled onset using one-to-one matching. Release timing is excluded because physical releases are not measured.

Source Modules

Source file	Main responsibility
piano_handtracking.py	Main coordinator, load calibration, track hands, coordinate analysis and output
piano_calibration.py	Load JSON or collect corner and octave anchor calibration
piano_keyboard.py	Perspective mapping, local key spacing, fingertip lookup, and key outlines
piano_press.py	Visual candidate counts and audio confirmation coordination
piano_sound.py	Audio extraction via FFmpeg, librosa onset detection, and FFT pitch checks
piano_notes.py	Format confirmed note events and write performance.mid using pretty_midi
piano_video.py	Render overlays, landmarks, and key outlines onto intermediate video

Note states

Physical key releases are not measured. Detected note onsets use a 0.20-second placeholder duration for MIDI and display purposes, which is shortened when the same pitch starts again.

Evaluation Methodology

Reported accuracy measures exact pitch matches within ±0.10 seconds of a labeled onset, using one-to-one matching. Release timing is excluded because physical releases are not measured.

Result	Counting criteria
TP	A correct note is detected within the time tolerance.
FP	An extra or incorrect note was counted.
FN	An actual note was missed.

Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
F1 = 2 x Precision x Recall / (Precision + Recall)

A wrong note detection can count as both a false positive and a false negative. Evaluation counts individual notes rather than counting every video frame as a new note. Hovering-only tests were performed on an earlier prototype. Because the detection approach has changed substantially, a dedicated hovering-only test and measurement of false detections per minute remain future work for the current model.

Test Cases

Recording Types	What it checks
White and black keys	Correct pitch and key mapping across the played range.
Repeated notes	Separate presses and releases without merged or duplicate notes.
Hovering (without playing)	False note triggers caused by hand position or shadows.
Simple melodies and 3 note chords	Timing and simultaneous notes. Chords remain an optimal goal.

Seven recordings were evaluated. These recordings also informed tuning, so the reported scores are development results. Evaluation on separate unseen recordings remains future work. Fingertip localization accuracy and physical release timing were not evaluated.

Across seven recordings, the program processed 3,436 frames in 209.91 seconds, averaging 16.37 FPS. Six of seven recordings met the 15 FPS throughput target. These measurements include analysis and output generation. Pooled throughput is total frames divided by total processing time.

Performance Evaluation and Results

Benchmark tests were evaluated under Windows 11 and VMware Ubuntu environments on a PC using an AMD Ryzen 7 5700 processor and 16 GB RAM. Dependencies include Python, OpenCV, MediaPipe, NumPy, librosa, pretty_midi, and imageio-ffmpeg. Processing uses CPU execution.

Recording Results

Recording	Frames	Process Time	FPS
C4 C-major scale, one hand	386	24.42 s	15.81
C-major scale, both hands	254	15.92 s	15.96
C5 repeated eight times	619	35.76 s	17.31
Black-key scale	311	20.61 s	15.09
Full mixed performance	1,170	68.65 s	17.04
Twinkle Twinkle Little Star	211	14.77 s	14.29
Unassigned source entry	492	Not reported	Not reported
Combined	3,436	209.91 s	16.37

Across the combined test suite of 3,436 frames processed over 209.91 seconds, the system averaged 16.37 FPS, with six of seven clips meeting or exceeding the 15 FPS minimum throughput goal.

Frame-count discrepancy: The six named recordings total 2,951 frames. The source table also places a 492-frame value in its Combined row. If 492 represents a seventh recording, the individual counts total 3,443 frames, seven more than the reported aggregate of 3,436. The 492-frame value is preserved above as an unassigned source entry; its recording identity, processing time, and FPS are not reported.

The corrected aggregate result is six of seven recordings meeting the 15 FPS throughput target. The available individual rows do not independently verify that count. No individual measurements have been inferred or replaced.

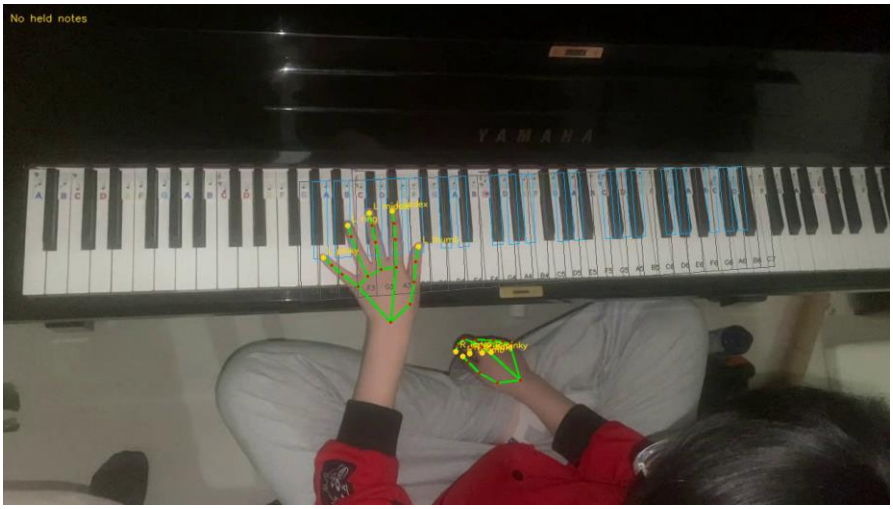
Other Performance Measurements

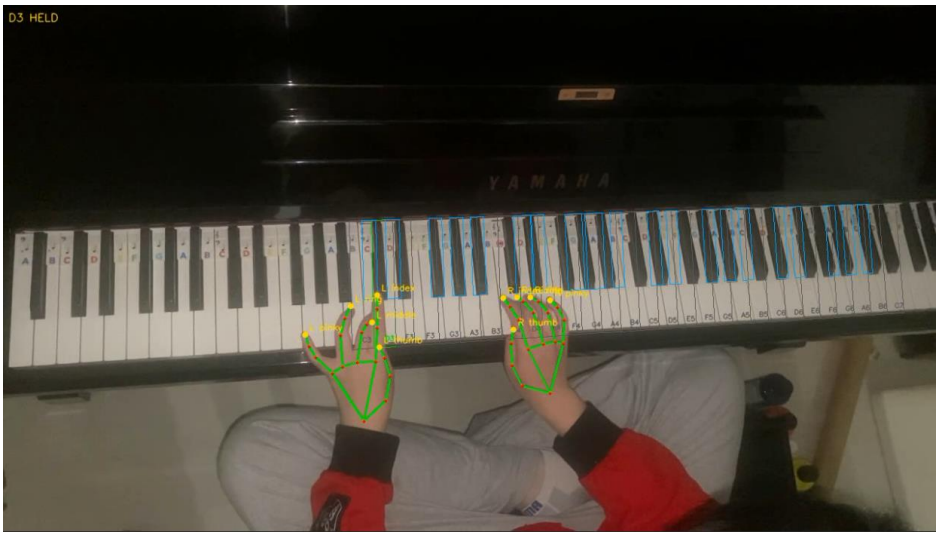
The program prints detected note count, total processing time including video output, and overall processing FPS. A full test run recorded Python process memory of approximately 343.5 MiB, excluding separate FFmpeg processes. Application startup time and live capture to output delay remain unmeasured.

Proof of Concept and Remaining Challenges

The program produces annotated video at 1280 by 720 pixels for the prepared test recordings. OpenCV draws the calibrated keyboard outlines, hand connections, fingertip dots, and detected note names. These overlays show how the tracked fingertips relate to the keyboard map.

Camera movement, hidden fingertips, and overlapping notes can still cause missed or extra detections. Physical key releases and playing dynamics (velocity/intensity) are not measured. MIDI notes use estimated durations of up to 0.20 seconds and a fixed velocity of 64.





When the program runs, it saves annotated.mp4 and performance.mid in the selected results directory. The annotated video includes audio from the input recording, while the MIDI contains the detected pitches, start times, and estimated note durations.

Conclusion and References

Piano Analyzer combines manual keyboard mapping, two hand tracking, and audio analysis to produce MIDI and annotated video. Across seven different recordings, it achieved 87.74% precision, 84.19% recall, and an F1 score of 85.93% at 16.37 FPS pooled processing throughput. The target precision and recall thresholds were met in aggregate, but the original 30 FPS goal was not reached. Future work includes physical key release detection, MIDI velocity estimation, and improved accuracy and testing on difficult passages.

References

J. Sanchez. Earlier hand-tracking prototype report, 2026. Related starter files include roadway_detection.cpp, handgestureimproved.cpp, and handgesturemediapipe.py.

Google. Gesture recognition guide for Python.
https://developers.google.com/edge/mediapipe/solutions/vision/gesture_recognizer/python

OpenCV. Geometric Image Transformations.
https://docs.opencv.org/4.x/da/d54/group__imgproc__transform.html

OpenCV. Getting Started with Videos.
https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html

librosa development team. librosa.onset.onset_detect Documentation.
https://librosa.org/doc/latest/generated/librosa.onset.onset_detect.html

NumPy Developers. Fast Fourier Transform (numpy.fft).
https://numpy.org/doc/stable/reference/routines.fft.html

pretty_midi development team. pretty_midi Documentation.
https://craffel.github.io/pretty-midi/

MIDI Association. Standard MIDI Files Specification.
https://midi.org/standard-midi-files-specification

FFmpeg. ffmpeg Documentation.
https://ffmpeg.org/ffmpeg.html

CannyCam example source (cannycam.cpp). External code incorporated into the hand-tracking starter.

OpenAI. ChatGPT conversation on Piano Analyzer development and implementation assistance.
https://chatgpt.com/share/6acdb5d3-832c-83e8-b40c-f1add5a9f94b

Attributions

The hand-tracking starter was adapted from Google's MediaPipe example and the external CannyCam example (cannycam.cpp). ChatGPT assisted with calibration, audio analysis, MIDI encoding, and two-hand tracking. A pianist helped perform the recordings.

Appendix Build Run and Supporting Files

The project source folder contains seven Python modules: piano_handtracking.py, piano_calibration.py, piano_keyboard.py, piano_press.py, piano_sound.py, piano_notes.py, and piano_video.py. A Makefile provides syntax checks and a default test run.

Required dependencies include Python, OpenCV, MediaPipe, NumPy, librosa, pretty_midi, and imageio-ffmpeg. The file models/gesture_recognizer.task supplies the pretrained MediaPipe model used for hand landmark tracking.

Testing Windows Commands

To run the program, you must run this command from the project folder. You can either use a new or an empty output folder.

```
python piano_handtracking.py --input
"videos\c5_repeated_8_times\c5_repeated_8_times.MOV" --calibration
"calibration\c5_repeated_8_times\keyboard.json" --output
"results\c5_repeated_8_times" --show
```

The --show option displays the annotation preview if you want to see it. Add --calibrate to manually select keyboard corners and C-key anchors when a new keyboard map is needed.

After processing, the terminal prints the detected-note count, total runtime including video output, and overall processing FPS.

Makefile for Ubuntu

```
make check PYTHON=python3
make run PYTHON=python3 FLAGS=--show
```

The Makefile checks all seven Python files and enables preview only when --show is supplied.

Supporting Files

The project archive contains the seven source code .py files, Makefile, MediaPipe model, input recordings, reference MIDI files, and saved keyboard calibrations. Reference MIDI files were created manually from the video recordings and compared with the .mp4 files in a DAW.

The results folder is initially empty. Running the program creates the selected output folder and saves performance.mid and annotated.mp4. Intermediate audio and silent video files are stored temporarily and removed after processing.